

RENSSELAER MECHATRONICS

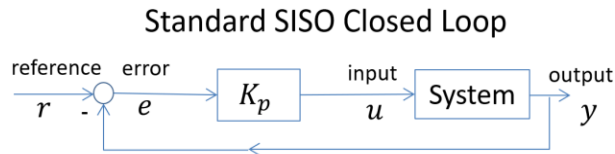
DC Motor Control with SISOtool: Proportional Control

Objectives:

- Design a Proportional controller with SISOtool to meet transient specifications
- Based on this design determine the reference value of a step that will avoid saturation
- Determine the expected steady state error
- Compare the experimental response to the predicted response
- Improve the experimental response by cancelling out coulomb friction using feedback linearization

Background Information:

The standard SISO controller uses the output compared to a reference.



For a proportional controller the control law $u = K_p e$ can be used to determine the maximum reference size to avoid saturation. If the proportional gain K_p is known, initial output is assumed to be zero $y = 0$, and the maximum control effort is known u_{max} then

$$u = K_p e = K_p (r - y) \rightarrow u_{max} = K_p r_{max} \rightarrow r_{max} = \frac{u_{max}}{K_p}$$

The control law can also be used to determine the expected steady state error if we know the coulomb friction causes the motor to stop at a specified voltage. In this case if the coulomb voltage is known $u_{coulomb}$ and the proportional gain is known K_p then the expected error can be calculated:

$$u = K_p e \rightarrow u_{coulomb} = K_p e_{ss} \rightarrow e_{ss} = \frac{u_{coulomb}}{K_p}$$

Part 1: Proportional control with SISOtool:

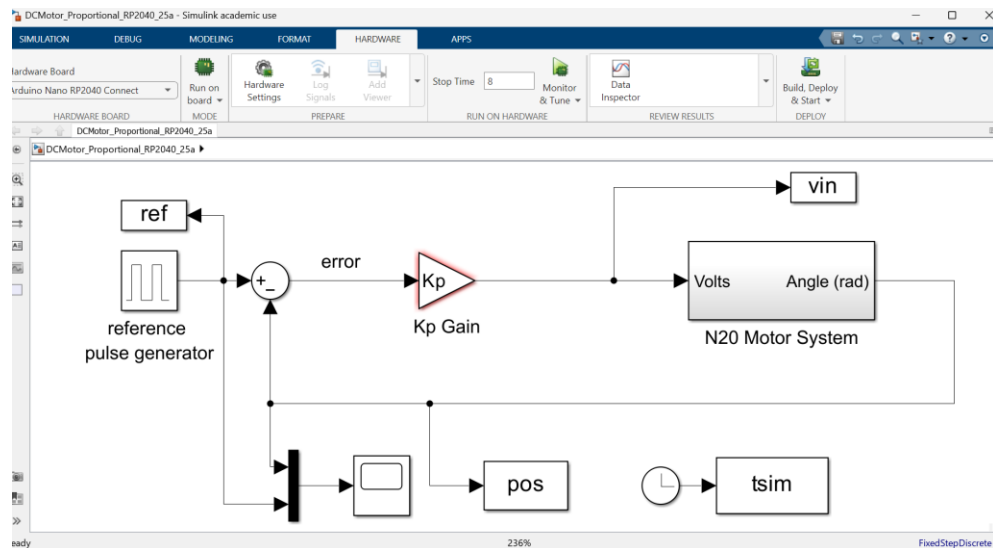
Design a standard SISO controller (using SISOTOOL in MATLAB) for the position control of a DC motor such that the closed loop poles satisfy 5% overshoot for a unit step response. Assume a DC motor model with $K = 5.5 \frac{\text{rad/sec}}{\text{v}}$, $\tau = 0.045 \text{ sec}$.

Questions:

- What is the value of the proportional gain to achieve 5% overshoot?
- For this value of gain calculate the maximum reference r_{max} if the maximum voltage is 4.65 volts.
- For this gain calculate the expected steady state error e_{ss} if the minimum voltage for the motor to overcome coulomb friction is 0.5 volts.
- Where are the closed loop poles and zeros located (click on the closed loop poles in SISOtool it will be displayed in the bottom right)
- Include a screenshot of your SISOtool that shows the control effort and response for your designed proportional gain and calculated r_{max}

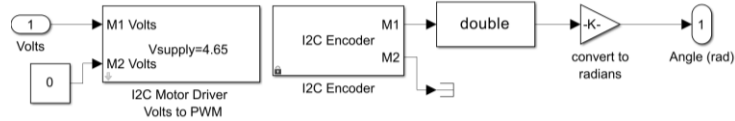
Hardware Verification:

Create the following Simulink diagram:

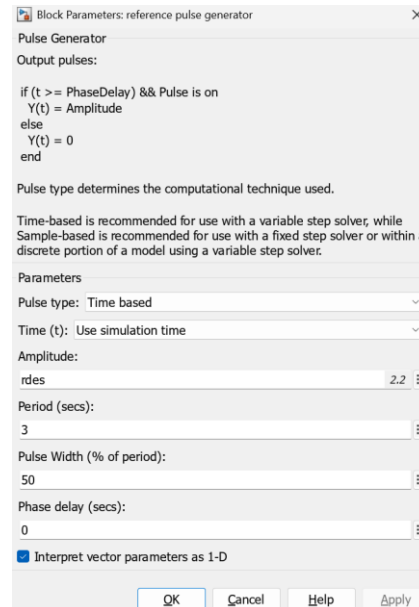


- Use a fixed step discrete solver with a sample time of 0.005 seconds and a stop time of 8 seconds.

The motor subsystem is composed of the following:



The pulse block uses a variable 'rdes' as the specified reference value, a period of 3 seconds, and a pulse width of 50%. This means a stop time of 8 seconds will capture a couple step responses:



Next create a script file that will define the reference, controller, run and plot the results:

```
%% Proportional Control:
rdes=1; % desired reference value
Kp=1; % desired Kp

%% Open the simulink diagram and run experiment:

%% Plot Results:
close all
plot(tsim, pos), hold on
plot(tsim, ref, 'r-')

% Use logged simulation time and reference
% to calculate predicted response based on model.
% plant transfer function G needs to be defined
[y t]=lsim(feedback(Kp*G,1),ref,tsim);
plot(t,y, 'k--')
legend('experiment','reference','theory')
```

Evaluate the first part to define the experiment parameters. Run the Simulink diagram. Then evaluate the last part to plot the results and compare the results to the theory. The code does the following:

- Defines desired reference 'rdes' and controller 'Kp' (do this before running the Simulink diagram)
- Plots the experiment results

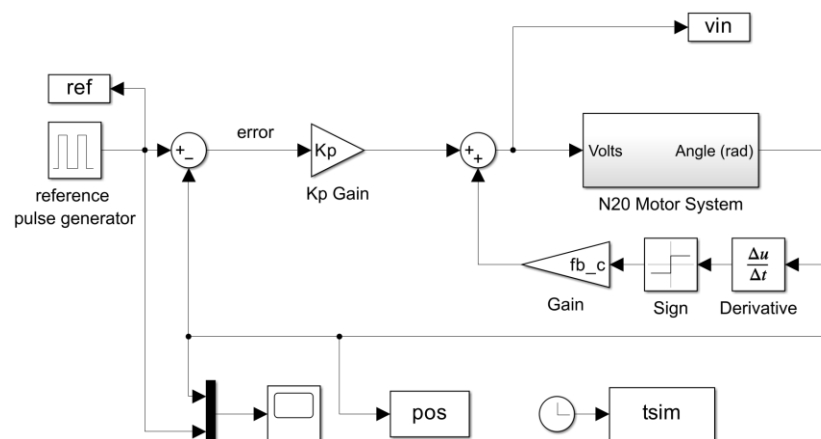
- Uses the stored simulation reference value 'ref' and time 'tsim' to simulate what the theoretical results are based on the system model transfer function 'G' (G should be defined already from your sisotool design). The command 'feedback(Kp*G,1)' calculates the closed loop feedback system transfer function to use in the 'lsim' command. This way it can compute the theoretical result as the reference change up and down from the pulse.

Questions:

- Use the code above to run your proportional controller. Include two graphs. One graph showing the complete 8 seconds. And a second graph that is zoomed in on the first step change at time 0. You should notice the response will match initially but will not have any overshoot due to the coulomb friction.

Part 2: Addressing Coulomb friction with Feedback linearization

A voltage estimate of the coulomb friction is estimated and added to the input voltage:



The script file is used to define the coulomb friction voltage:

```

%% Proportional Control:
rdes=1;    % desired reference value
Kp=1;      % desired Kp
fb_c=0.4;  % coulomb friction voltage estimate

%% Open the simulink diagram and run experiment:

%% Plot Results:
close all
plot(tsim, pos), hold on
plot(tsim, ref, 'r-')

% Use logged simulation time and reference
% to calculate predicted response based on model.
% plant transfer function G needs to be defined
[y t]=lsim(feedback(Kp*G,1),ref,tsim);
plot(t,y, 'k--')
legend('experiment', 'reference', 'theory')

```

You will need to run several tests to find a good value of 'fb_c' such that the experimental overshoot matches the expected overshoot. It will probably be in the range of 0.35 to 0.5 depending on your motor. Find a value such that the percent overshoot matches well, but not so high that the system starts having additional oscillations.

You will also notice that there will still be steady state error within the value you calculated in the first section. This will be within the dead band of the coulomb friction area so we expect it will stop - at the peak of the overshoot the velocity is zero so it will not provide feedback linearization. This is why cancelling out the coulomb with a feedforward term might be more successful because if a desired velocity profile is specified, we would know how to cancel out the coulomb friction based on the input instead of an estimation based on the measured velocity.

Questions:

- Use the code above to run your proportional controller with feedback linearization. Include two graphs. One graph showing the complete 8 seconds. And a second graph that is zoomed in on the first step change at time 0. You should notice the response should match the predicted response pretty well in terms of % overshoot but will likely still have steady state error within the range predicted in the first section.